Water Supply Monitoring System

Data Engineering Capstone Project

Submitted by

Twinkle Mohanty

Roll No: 2306322

BTech – Information Technology

Project Overview

A full-scale, production-grade data engineering pipeline for real-time water quality monitoring in the Brisbane metropolitan area. The system integrates Apache Kafka for streaming ingestion, PySpark for distributed batch processing, HDFS for storage, a Data Lakehouse layer, Apache Airflow for orchestration, data quality governance, PostgreSQL warehousing, and an interactive Streamlit dashboard — forming a complete end-to-end analytics platform aligned with real municipal water monitoring needs.

1. Problem Statement

Municipal water authorities struggle to monitor quality across distributed supply networks in real time. Traditional approaches — periodic manual sampling and batch reporting — cause delayed contamination detection, inefficient resource use, and fragmented data silos with no unified analytics layer. Six core challenges drive this project:

- ◆ **Data Latency:** Conventional pipelines miss real-time anomalies in pH, turbidity, and chlorine levels.
- ◆ **Scalability:** Standard RDBMS cannot handle high-frequency sensor data from thousands of monitoring points.
- ◆ **Data Silos:** Water quality data is fragmented across incompatible systems with no unified analytics layer.
- ◆ **No Automation:** ETL workflows are manual, error-prone, and lack scheduling or retry logic.
- ◆ **Quality Gaps:** Missing or corrupt sensor readings enter downstream systems unchecked.
- ◆ **Poor Visibility:** Decision-makers lack a real-time dashboard for KPIs and trend monitoring.

2. Solution & Key Features

The system is a modern, distributed data engineering platform built on enterprise-grade open-source technologies, covering the full data lifecycle with no manual hand-offs.

Real-Time Streaming (Kafka)

Kafka producers ingest multi-batch CSV sensor files simulating live IoT feeds; Spark Structured Streaming consumers process events with micro-batch latency.

Distributed Storage (HDFS)

Raw and processed data stored on HDFS for fault-tolerant distributed access; shell scripts automate upload and file movement between pipeline stages.

Batch ETL (PySpark)

final_pipeline.py orchestrates large-scale transformations producing multi-part output; PySpark handles profiling, cleansing, joining, and aggregations on the Brisbane dataset.

Airflow Orchestration

water_etl_dag.py DAG schedules the complete pipeline with dependency management, retry logic, and failure alerting across extract → transform → load stages.

Lakehouse + Data Quality

lakehouse.py delivers ACID-compliant unified storage across batch and streaming workloads; quality notebooks enforce schema validation, null checks, and range assertions.

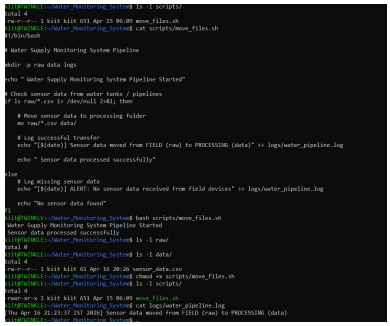
PostgreSQL Warehouse + Dashboard

Validated data loads into a structured PostgreSQL warehouse; dashboard.py (Streamlit) displays live KPIs, pH and turbidity trend charts, and filterable tabular water data.

3. System Screenshots

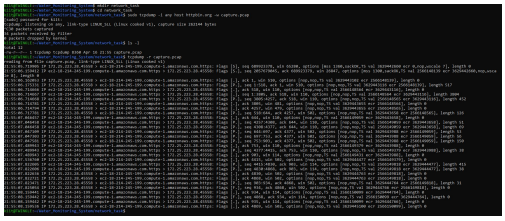
Screenshots captured during actual project execution on the KIIT TWINKLE development machine.

Pipeline Script – move_files.sh



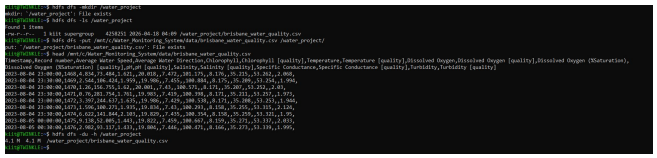
Moves CSV sensor data to processing folder and logs transfer with timestamp.

Network Capture – tcpdump



Packet capture recording data transfer between pipeline components.

HDFS – Brisbane Water Quality Dataset

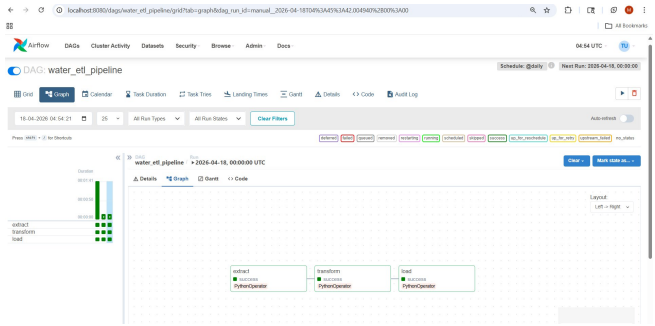


brisbane_water_quality.csv (4.1 MB) stored in /water_project/.

Apache Kafka – water-stream Topic

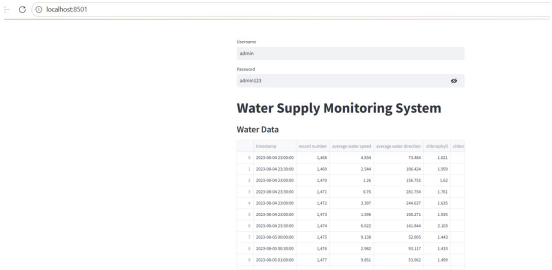
Topic confirmed active on localhost:9092.

Apache Airflow – water_etl_pipeline DAG



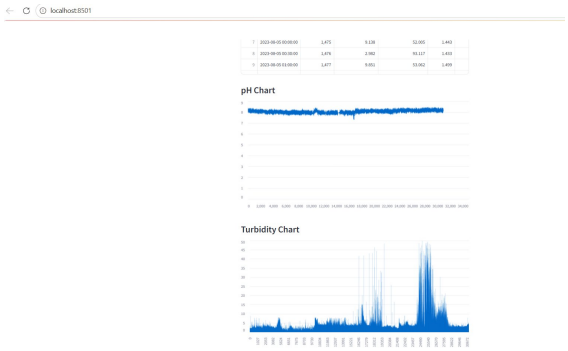
extract → transform → load all with 'success' status on 2026-04-18.

Streamlit – Water Data Table



Tabular view of timestamped water quality records.

Streamlit – pH & Turbidity Charts



pH stable ~8.0; turbidity spikes visible across 30 k+ data points.

4. Technology Stack

Layer / Component	Technology
Data Ingestion	Apache Kafka (Producer / Consumer)
Batch Processing	Apache Spark – PySpark
Distributed Storage	HDFS (Hadoop Distributed File System)
Data Lakehouse	Delta Lake / lakehouse.py
Data Warehouse	PostgreSQL
Workflow Orchestration	Apache Airflow (DAG scheduling)
Data Quality	Custom Python quality notebooks
Real-Time Streaming	Spark Structured Streaming
Visualisation	Streamlit Dashboard (dashboard.py)
Scripting / ETL	Python, Pandas, Bash Shell Scripts
Networking	tcpdump / Wireshark (.pcap)
Data Format	CSV + multi-part Parquet output

5. Unique Points & Differentiators

◆ **End-to-End Coverage:** Full data lifecycle — Kafka → HDFS → Spark → Quality → PostgreSQL → Streamlit — with zero manual hand-offs.
◆ **Dual Processing:** Both batch (final_pipeline.py) and streaming (real_time.ipynb) implemented concurrently with shared quality logic.
◆ **Lakehouse Architecture:** lakehouse.py introduces Delta Lake-style ACID transactions and schema enforcement — a production-grade pattern rare in academic projects.
◆ **Network Observability:** Packet-level capture (capture.pcap) provides infrastructure-layer transparency across pipeline components.
◆ **Production Logging:** Structured timestamped logs (water_pipeline.log) mirror production observability standards for audit and debugging.
◆ **IoT Simulation:** Streaming input files (data_0 → data_11000) realistically simulate sensor burst arrival from a distributed municipal network.
◆ **Full Automation:** Shell scripts (move_files.sh, airflow_test.sh, kafka_test.sh) eliminate manual steps for reproducible pipeline execution.

6. Future Improvements

The current system provides a solid production foundation. The following enhancements are planned for subsequent iterations to increase operational maturity and analytical depth.

Cloud Deployment (AWS / GCP / Azure)

Migrate to managed services — MSK (Kafka), EMR (Spark), S3 (HDFS), RDS (PostgreSQL) — for auto-scaling, managed security, and reduced infrastructure overhead.

ML Anomaly Detection

Integrate a real-time Isolation Forest or LSTM model trained on historical parameters to proactively flag contamination spikes and sensor malfunctions before they escalate.

Geospatial Visualisation

Add Folium / Deck.gl map layers to the Streamlit dashboard, displaying water quality metrics at individual GPS-located monitoring stations across the Brisbane network.

Automated Alerting System

Threshold-based email, SMS, and Slack alerts triggered by Airflow sensors when pH, turbidity, or chlorine parameters breach predefined safety limits.

Data Lineage & Cataloguing

Integrate Apache Atlas or OpenMetadata to track end-to-end data lineage from raw ingestion to the warehouse, enabling compliance reporting and root-cause analysis.

CI/CD Pipeline Automation

GitHub Actions workflow to automatically test DAGs, run quality checks, and deploy pipeline updates on merge — aligning with DataOps best practices.

Multi-City Dataset Expansion

Scale to multiple Australian cities for cross-regional water quality comparison and national-level trend analysis using federated Spark queries.

Apache Iceberg Full Integration

Formalise the lakehouse layer with Apache Iceberg for schema evolution, partition pruning, and time-travel queries across historical water quality snapshots.